Choosing a cross-platform technology for a Hangman app

Tri Le

October 2026

What the application has to do

The application lets one person play Hangman on a phone. It picks a word, hides the letters, takes guesses one letter at a time and ends when the word is revealed or the attempts run out. Two things in the brief shape the technology decision more than the game does. The solution has to support Norwegian and English in full, including an info page in the selected language that is reachable both at startup and during play, and the drawing of the gallows grows with every wrong guess, so the app repaints a shape of its own rather than only arranging text and buttons.

Those two requirements pull in different directions. Translating an interface is mostly a question of how well a framework handles resources, locale resolution and plural forms. Repainting a drawing is a question of how the framework gets pixels onto the screen. A fair comparison has to score the candidates on both, because this app is small enough to be built in any of them and the exercise is about which one is defensible rather than which one is quickest to start.

The candidates

Ionic builds the interface from web standards. Its components are "built with web standards using HTML, CSS, and JavaScript" and the app ships inside a platform web view: "Both iOS and Android software development kits (SDKs) provide Web Views which render any Ionic app, while still allowing for full Native SDK access" (Ionic, n.d.). The application code is Angular, React or Vue, so the language is JavaScript or TypeScript. Native device access goes through Capacitor.

Apache Cordova is the older member of the same family. It "allows you to use standard web technologies - HTML5, CSS3, and JavaScript for cross-platform development", and "the Cordova-enabled WebView may provide the application with its entire user interface" (Apache Cordova, n.d.). Device features arrive as plugins, and a new project ships with none: "Any plugins you desire, even the core plugins, must be explicitly added" (Apache Cordova, n.d.). Ionic with Capacitor is the maintained path to the same architecture, which makes Cordova hard to argue for in a new project.

React Native writes the interface in JavaScript or TypeScript with React, and renders real platform views rather than a web view. Its current architecture removes the old serialising bridge: "The New Architecture removes the asynchronous bridge between JavaScript and native and replaces it with JavaScript Interface (JSI). JSI is an interface that allows JavaScript to hold a reference to a C++ object and vice-versa. With a memory reference, you can directly invoke methods without serialization costs" (Meta, n.d.). The renderer is Fabric and the JavaScript engine is Hermes. The UI is native, but the code that drives it still runs in a JavaScript runtime.

Flutter writes the interface in Dart and draws it itself. "Flutter has its own implementations of each UI control, rather than deferring to those provided by the system", and it "minimizes those

abstractions, bypassing the system UI widget libraries in favor of its own widget set" (Google, n.d.). Release builds are not interpreted: "For release, Flutter apps are compiled directly to machine code, whether Intel x64 or ARM instructions, or to JavaScript if targeting the web" (Google, n.d.). Rendering goes through Impeller, which "is shipped along with the application" (Google, n.d.).

Kirigami is KDE's framework for convergent applications, "built on top of the QML language and Qt Quick Controls 2 components provided by the Qt project", and it "works on a variety of platforms, such as Plasma Mobile, Desktop Linux, Android, MacOS, and Windows" (KDE Community, n.d.). An application is QML with C++ behind it, so two languages, and the Android story is the least travelled of the five.

Internationalisation

Flutter treats translation as a first-party concern. Strings live in ARB files, one per locale, and the toolchain generates a typed accessor class from them, with plural and placeholder syntax built into the format. The framework resolves the locale from the device and accepts an explicit override, and the generated class exposes the list of supported locales so the two cannot drift apart. In this app that machinery carries not only the labels but the alphabet itself, which is why the Norwegian keyboard has 29 keys, ending in æ, ø and å, while the English one has 26.

React Native has no equivalent in the framework. Translation comes from a library chosen per project, most often i18next with a locale module, which works but leaves the format, the plural rules and the fallback behaviour as decisions the developer has to make and keep consistent. Ionic inherits whatever the chosen web framework provides, and the web ecosystem is rich here, so the capability is real but not uniform. Cordova offers nothing and the app would carry its own solution. Kirigami relies on Qt's translation system, which is mature, though the tooling is aimed at desktop workflows.

For a translation-shaped app the ranking is Flutter first on integration, Ionic and Kirigami close behind on capability, React Native workable, Cordova last.

Fast screen updates

The order changes when the screen has to be redrawn quickly. A web view renders through a browser engine, so an animation competes with layout, style recalculation and the single JavaScript thread that drives it. That is the weak point of both Ionic and Cordova, and it is the reason neither would be my choice for an action game. React Native is a step up because the views are native, and JSI removed the serialisation that used to make every frame expensive, but the application logic still executes in a JavaScript runtime. Kirigami sits on Qt Quick, which has a proper scene graph and is genuinely fast, at the cost of QML plus C++ and the weakest Android support here. Flutter compiles to machine code and owns the pipeline end to end, which is why it is the one candidate that is strong at this pole without giving up the other.

Hangman does not need any of that. Six redraws of a stick figure is nothing. The point is that the choice should hold if the next app in the same stack is a game, and only one candidate scores well on both requirements at once.

Language, typing and compilation

Dart is statically typed with sound null safety, and release builds are compiled ahead of time to ARM machine code. TypeScript on React Native or Ionic gives static types at authoring time over a dynamically typed runtime, so a type error that escapes the compiler surfaces on the device instead.

QML is dynamically typed with the typed part of the program living in C++, which splits one application across two languages. Plain JavaScript on Cordova has neither types nor compilation.

For an exercise where the grader runs the code on an emulator, errors caught at compile time are worth more than errors caught by a user. That argument favours Dart, and Kirigami's C++ half only after accepting a second language.

The choice

I built the application in Flutter. It is the only candidate that is strong on both of the axes the brief creates: first-party internationalisation with generated, typed string access, and an owned rendering pipeline compiled to machine code for the drawing. Dart's static typing and ahead-of-time compilation move errors from runtime to build time, and one widget set means the Norwegian and English layouts, including a keyboard whose alphabet changes with the language, come out of the same code rather than two platform variants.

The honest cost is that Flutter is a separate SDK and Dart is a language I had not written before this project, so the toolchain install and the ramp were real work that React Native would not have cost me, since I already write TypeScript. That is the part of the trade I want on the record: the decision was made against the requirements and not against my familiarity, and it went against my familiarity.

React Native is the runner-up, and it would be a reasonable answer. It renders native views, JSI has removed the old bridge cost, and TypeScript is a serious language. It loses on two points. Its internationalisation is a third-party decision rather than a framework guarantee, and its logic runs in a JavaScript runtime, which is the layer I would have to reason about first if this app ever needed to animate at a steady frame rate. Ionic and Cordova lose on rendering, and Cordova again on being superseded by Capacitor. Kirigami loses on the two-language split and on Android being its least supported target.

References

Apache Cordova. (n.d.). *Overview*. Retrieved 12 October 2026, from https://cordova.apache.org/docs/en/latest/guid

Google. (n.d.). *Flutter architectural overview*. Retrieved 12 October 2026, from https://docs.flutter.dev/resources/arc overview

Ionic. (n.d.). *Fundamentals*. Retrieved 12 October 2026, from https://ionicframework.com/docs/core-concepts/fundamentals

KDE Community. (n.d.). *Kirigami*. Retrieved 12 October 2026, from https://community.kde.org/Kirigami

Meta. (n.d.). *About the New Architecture*. Retrieved 12 October 2026, from https://reactnative.dev/architecture/land page